

DAY 1

Image Processing Foundations

& The World of Computer Vision

How computers see • Pixels & color spaces • Image operations • Real-world applications

DAY 1

Image Processing
Fundamentals

TODAY

DAY 2

Edge Detection &
Feature Extraction

DAY 3

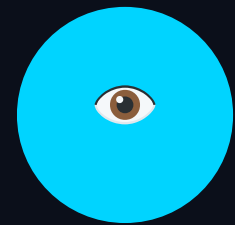
Video Analysis &
Object Tracking

DAY 4

Deep Learning
for CV (YOLO)

DAY 5

Deployment &
Project Submission



What is Computer Vision?

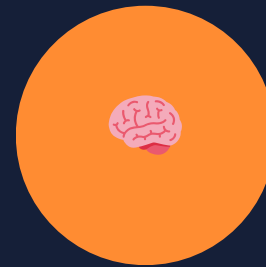
Teaching machines to interpret and understand visual information — images, videos, and live camera feeds.



Image Processing

Transform images: sharpen, filter, resize, denoise

Input: Image
Output: Better Image



Computer Vision

Understand images: detect objects, track motion, classify

Input: Image
Output: Meaning



Machine Learning

Learn patterns from data: CNNs, YOLO, Transformers

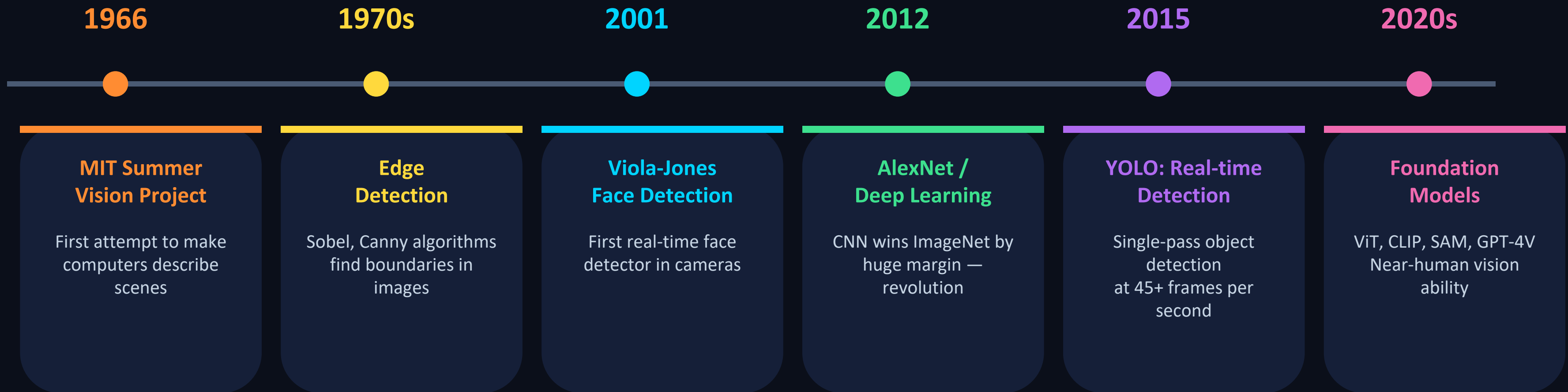
Input: Data
Output: Predictions

In Practice: All Three Work Together

Image Processing cleans the data → Machine Learning finds patterns → Computer Vision delivers understanding



The Evolution of Computer Vision



Key Insight

Before 2012, CV was hand-crafted rules. After 2012, deep learning made machines truly learn to see. The field is now a \$20+ billion industry.



How Computers See: Pixels & Images

To us, a photo is a scene. To a computer, it's a giant grid of numbers.

Grayscale Image

Each pixel = 1 number (0–255)

0 = Black 255 = White

0	50	100	150
50	100	150	200
100	150	200	250
150	200	250	255

Color Image (RGB)

Each pixel = 3 numbers (R, G, B)



Red (255, 0, 0)



Green (0, 255, 0)



Blue (0, 0, 255)



Yellow (255, 255, 0)



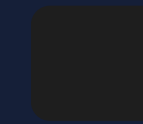
Orange (255, 165, 0)



Purple (128, 0, 255)



White (255, 255, 255)



Black (0, 0, 0)

Image = 3D Array in Memory

A 1080p color image has shape (1080, 720, 3) = Height × Width × Channels (R,G,B). That's over 6 million numbers. For video at 30 fps, that's 186 million numbers per second. This is why efficient processing matters!



Color Spaces — Different Lenses for Analysis

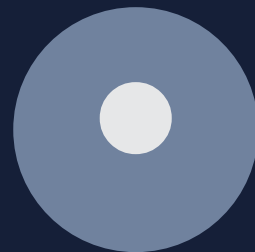


RGB

Red, Green, Blue

How screens display color.
3 channels, 0–255 each.
16.7 million possible colors.

Best for: Display, general use



Grayscale

Single Channel

One value per pixel (0–255).
3× less data than RGB.
Edges are intensity changes.

Best for: Edge detection, speed



HSV

Hue, Saturation, Value

Separates color type from
brightness. Robust to lighting.
Hue = color wheel position.

Best for: Color-based detection



LAB

Lightness, A, B

Perceptually uniform — equal
numeric change = equal visual
change to human eyes.

Best for: Color correction

Why HSV Matters for CV

Problem: Detecting "red" in RGB is hard — (255,0,0), (150,20,20), (255,100,50) are all "red" but very different numerically.

Solution: In HSV, red is simply a range of Hue values, regardless of brightness. One filter catches all reds. Convert with: `cv2.cvtColor(image, cv2.COLOR_BGR2HSV)`



Essential Image Operations



Read & Display

```
cv2.imread('img.jpg')  
cv2.imshow('Win', img)  
cv2.imwrite('out.jpg', img)
```



Resize

```
cv2.resize(img, (640,480))  
INTER_AREA for shrink  
INTER_CUBIC for enlarge
```



Crop

```
roi = img[y1:y2, x1:x2]  
Simple array slicing!  
No function needed
```



Rotate & Flip

```
cv2.flip(img, 1) horizontal  
cv2.flip(img, 0) vertical  
getRotationMatrix2D
```



Draw

```
cv2.rectangle() for boxes  
cv2.circle() for points  
cv2.putText() for labels
```



Brightness

```
cv2.add(img, 50) brighter  
cv2.subtract(img, 50) darker  
addWeighted() for blending
```

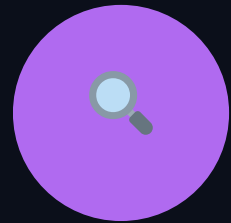


Image Filtering — Cleaning & Enhancing

Filtering applies a small matrix (kernel) to every pixel using its neighbors. This is the same convolution operation used in CNNs.

Gaussian Blur

Smooths with bell-curve kernel.
Closer pixels = more influence.
Removes fine noise.

```
cv2.GaussianBlur(img, (5,5), 0)
```

Most common preprocessing step

Median Blur

Replaces pixel with median of
neighbors. Excellent against
salt-and-pepper noise.

```
cv2.medianBlur(img, 5)
```

Best for impulse noise removal

Sharpening

Enhances edges and fine details.
Custom kernel emphasizes
center pixel vs neighbors.

```
cv2.filter2D(img, -1, kernel)
```

Makes features easier to detect

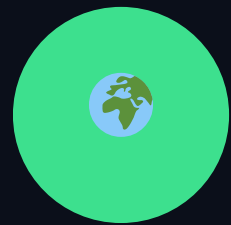
Denoising

Advanced noise removal using
non-local means algorithm.
Slow but very effective.

```
cv2.fastNlMeansDenoising(img)
```

Best quality noise removal

Rule of Thumb: Always blur before edge detection. Always denoise before classification. Preprocessing quality directly affects model accuracy.



Computer Vision Applications

A \$20+ billion industry transforming every sector



Healthcare

X-ray/CT analysis, tumor detection, retinal scans, cell counting

27% of CV spending



Autonomous Vehicles

Object detection, lane keeping, pedestrian safety, traffic sign reading

Life-critical decisions



Manufacturing

Defect inspection, quality control, assembly verify, barcode/QR scanning

35% of CV spending



Retail

Cashierless stores, visual product search, shelf monitoring, foot traffic

Fastest growing sector



Agriculture

Crop health monitoring, disease detection, fruit picking robots, weed ID

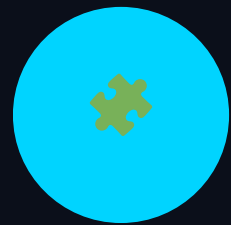
Addressing labor shortage



Aviation

Turnaround monitoring, runway FOD detection, passenger flow analysis

Our project domain!



The Core Tasks of Computer Vision

Every CV application is built from these fundamental building blocks

Classification

What is this?

"This image is a cat"

One label per image.
The simplest CV task.

Object Detection

What & Where?

"Cat at (x1,y1,x2,y2)"

Bounding boxes + labels.
Multiple objects per image.

Segmentation

Every pixel

Pixel-level labeling

Semantic: label all pixels.
Instance: separate objects.

Tracking

Follow over time

Object ID across frames

Links detections in video.
Handles occlusion.

Action Recognition

What's happening?

"Person is running"

Classifies activities.
Needs temporal context.

Pose Estimation

Body structure

Joint positions (x,y)

Skeleton detection.
Human pose & gestures.



Final Project: Airplane Turnaround Time

Build a CV system that analyzes airport gate video to calculate aircraft turnaround time

1

**Aircraft
Arrives**

In-Block

2

**Equipment
Approaches**

GSE Start

3

**Ground
Services**

Fuel, Board,
Baggage

4

**Equipment
Clears**

GSE End

5

**Aircraft
Departs**

Off-Block

**Turnaround Time = Off-Block Timestamp – In-Block
Timestamp**

Your System Must:

- Detect the aircraft and ground equipment
- Identify arrival and departure events
- Calculate total turnaround time
- Output a summary report with timestamps

Real-World Impact

Amsterdam Schiphol deployed "Deep Turnaround" using AI cameras for real-time gate monitoring. It improved on-time departures and reduced delays across the entire airport network.



Day 1 Key Takeaways

1

Images are grids of numbers

Grayscale = 2D matrix (H×W), Color = 3D array (H×W×3). Understanding this is understanding how computers "see."

2

Color spaces are your toolkit

RGB for display, HSV for color detection, Grayscale for speed. Choose the right lens for each task.

3

Preprocessing determines success

Blurring, denoising, resizing — these operations directly impact how well your models perform.

4

CV is transforming every industry

Healthcare, manufacturing, aviation, retail — the ability to extract meaning from pixels is now critical infrastructure.

5

Video = sequence of images

Everything you learn about single images applies to video. Process frame by frame at 30+ fps.

Tomorrow (Day 2): Edge Detection, Thresholding, Contours & Feature Extraction — the building blocks for object detection